



Universidad de El Salvador

Facultad Multidisciplinaria de Occidente - Ingeniería en Desarrollo de Software

Asignatura: Análisis y Diseño de Software I	Unidad 1	Tema: Modelo de procesos de desarrollo de software
	Semana 2	Tutor: Licdo. Edwin Aristondo Monzón González

Introducción:

Bienvenidos al material de lectura #2 de la Unidad #1 de la asignatura Análisis y Diseño de Software. En este material se estudiarán los modelos de procesos de desarrollo de software, su definición, clasificación e importancia dentro del ciclo de vida del software. Se analizarán los modelos en cascada, incremental, de prototipos y espiral, reconociendo sus fases, características, ventajas y desventajas, así como su aplicación según el tipo de proyecto.

El estudio de estos modelos permitirá comprender cómo se planifica, organiza y controla el desarrollo de un sistema, facilitando la selección del enfoque más adecuado en cada situación. Se te invita a participar activamente en las actividades propuestas para fortalecer tus conocimientos y desarrollar un aprendizaje significativo orientado al ámbito profesional.

[illegible]

Entonces Supongamos que estás desarrollando una aplicación para pedir comida en línea. En la fase de comunicación, se reúne con dueños de restaurantes para entender sus necesidades, el cómo manejan los pedidos y que problemas enfrentan. En la planeación, defines los módulos principales del sistema como la gestión de restaurantes, pedidos y pagos. Luego, en el modelado, creas diagramas de flujo y casos de uso que describen como los clientes hacen pedidos y como los restaurantes los procesan. En la construcción, programas las funcionalidades del sistema basadas en estos modelos, y finalmente, en el despliegue, la aplicación está disponible para los clientes y restaurantes que la usen.

Clasificación de los Modelos de Desarrollo

Estos modelos fueron diseñados para estructurar el desarrollo de software y reducir la incertidumbre.

- **1.2.2 Modelo Cascada:** Un enfoque secuencial donde cada fase debe completarse antes de pasar a la siguiente.

El modelo de la cascada, a veces llamado ciclo de vida clásico, sugiere un enfoque sistemático y secuencial para el desarrollo del software, que comienza con la especificación de los requerimientos por parte del cliente y avanza a través de planeación, modelado, construcción y despliegue, para concluir con el apoyo del software terminado. (Pressman, 2010, p.33)

Ventajas:

- Es fácil de comprender y aplicar.
- Presenta fases bien definidas, lo que mejora la planificación y el control.
- Produce documentación detallada en cada etapa.
- Permite estimaciones más precisas de tiempo y costo cuando los requisitos son estables.
- Es adecuado para proyectos con requisitos bien definidos

Desventajas

- menudo, es difícil para el cliente enunciar en forma explícita todos los requerimientos. El modelo de la cascada necesita que se haga y tiene dificultades para aceptar la incertidumbre natural que existe al principio de muchos proyectos.
- El cliente debe tener paciencia. No se dispondrá de una versión funcional del (de los) programa(s) hasta que el proyecto esté muy avanzado. Un error grande sería desastroso si se detectara hasta revisar el programa en funcionamiento.
- Entre los problemas que en ocasiones surgen al aplicar el modelo de la cascada se encuentran los siguientes: Es raro que los proyectos reales sigan el flujo secuencial propuesto por el modelo. Aunque el modelo lineal acepta repeticiones, lo hace en forma indirecta. Como resultado, los cambios generan confusión conforme el equipo del proyecto avanza.

Modelo de la cascada

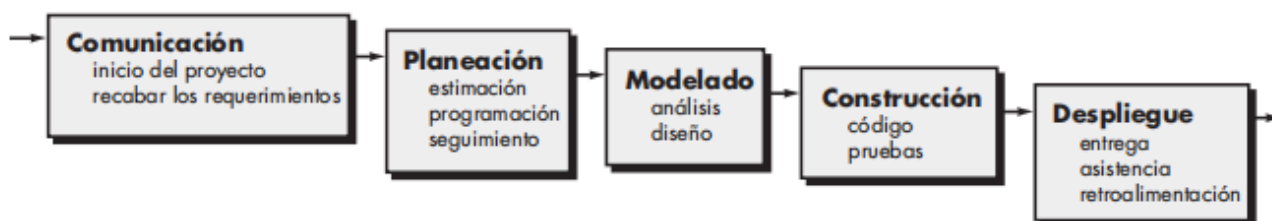


Figura 12

Modelo de la Cascada (Pressman, 2010, p.34)

- **1.2.3 Modelo Incremental:** Se desarrolla el software en módulos funcionales que se mejoran progresivamente.

Hay muchas situaciones en las que los requerimientos iniciales del software están razonablemente bien definidos, pero el alcance general del esfuerzo de desarrollo imposibilita un proceso lineal. Además, tal vez haya una necesidad imperiosa de dar rápidamente cierta funcionalidad limitada de software a los usuarios y aumentarla en las entregas posteriores de software. En tales casos, se elige un modelo de proceso diseñado para producir el software en incrementos.

El modelo incremental combina elementos de los flujos de proceso lineal y paralelo. El modelo incremental aplica secuencias lineales en forma escalonada a medida que avanza el calendario de actividades. Cada secuencia lineal produce "incrementos" de software susceptibles de entregarse de manera parecida a los incrementos producidos en un flujo de proceso evolutivo.

El modelo de proceso incremental se centra en que en cada incremento se entrega un producto que ya opera. Los primeros incrementos son versiones desnudas del producto final, pero proporcionan capacidad que sirve al usuario y también le dan una plataforma de evaluación.

- **1.2.4 El modelo de prototipos:** Es un enfoque de desarrollo de software en el cual se construye una versión preliminar del sistema

El objetivo de los prototipos es comprender los requerimientos del cliente y entonces desarrollar una definición mejorada de los requerimientos para el sistema. El prototipo se centra en experimentar con los requerimientos del cliente que no se comprenden del todo. El software se desarrolla para aprender más sobre un problema o explorar la factibilidad o la conveniencia de las posibles soluciones.

Un prototipo desechable no está pensado para ser utilizado como componente real del software que se entrega al cliente. Un prototipo no es un producto de calidad que deba mantenerse a largo plazo. Por el contrario, los prototipos son creados y probados rápidamente, para luego ser desechados. Sin embargo, es común que, por presiones de tiempo, se trate de enviar un prototipo al mercado como si éste fuera el producto final.

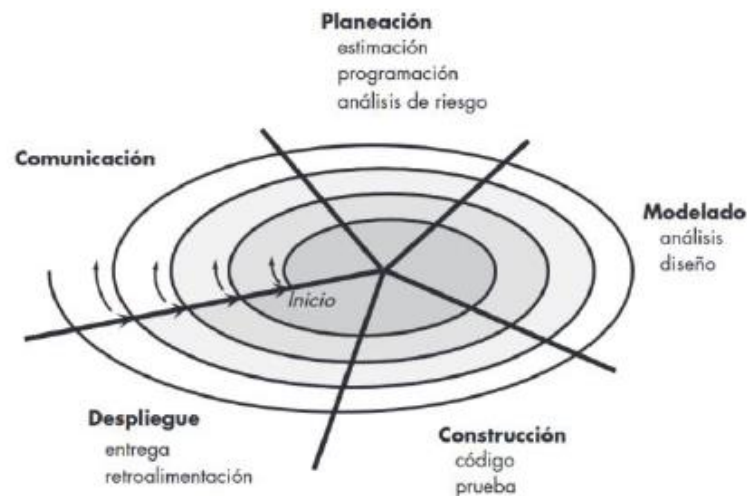
- **1.2.5 Modelo Espiral:** Se enfatiza la gestión de riesgos y la retroalimentación continua.

El Modelo de Espiral es un modelo de ciclo de vida orientado a los riesgos, dividiendo un proyecto software en miniproyectos. Cada miniproyecto se centra en uno o más riesgos importantes hasta que todos éstos estén controlados. El concepto "riesgo" puede referirse a requerimientos poco comprensibles, arquitecturas poco comprensibles, problemas de ejecución importantes, problemas con la tecnología subyacente, y demás.

El modelo en espiral combina una parte iterativa, que es la construcción de prototipos, con la parte secuencial del modelo en cascada para ir obteniendo resultados parciales en los que se tiene cierto control. Durante las primeras iteraciones, la versión incremental puede ser un modelo en papel o un prototipo.

Durante las últimas iteraciones, se producen versiones cada vez más completas del sistema.

Figura 1.3



Modelo de Espiral (Pressman, 2010, p.39)

1.2.6 Ventajas y desventajas del Modelo de procesos de Desarrollo de Software.

Ventajas

- Organización del trabajo: Permiten estructurar las actividades del desarrollo (análisis, diseño, codificación, pruebas, mantenimiento).
- Mejor control del proyecto: Facilitan la planificación, estimación de costos, tiempos y recursos.
- Seguimiento del progreso: Se puede medir el avance mediante fases, entregables o iteraciones.
- Documentación definida: Ayudan a mantener registros claros del sistema y del proceso.
- Mejora de la calidad: Incluyen etapas de verificación y validación para reducir errores.
- Comunicación entre el equipo: Todos conocen qué se debe hacer y en qué momento.

- Gestión de riesgos (en modelos como espiral): Permiten identificar problemas antes de que crezcan.
- Satisfacción del cliente (en modelos iterativos y ágiles): Se muestran avances frecuentes y se reciben retroalimentaciones.

Desventajas

- Rigidez en algunos modelos: En modelos tradicionales (como cascada) es difícil regresar a una fase anterior.
- Alta dependencia de los requisitos iniciales: Si están mal definidos, todo el proyecto se ve afectado.
- Exceso de documentación: Puede consumir tiempo y retrasar el desarrollo.
- Poca flexibilidad al cambio: Algunos modelos no se adaptan bien a nuevos requerimientos.
- Entrega tardía del producto (modelos secuenciales): El cliente ve el sistema funcionando hasta el final.
- Mayor costo si se detectan errores tarde: Corregir fallos en etapas avanzadas es más caro.
- No todos los modelos aplican a todos los proyectos: Elegir un modelo inadecuado puede afectar el éxito del sistema.

Bibliografía

Pressman, R. S. (2010). INGENIERÍA DE SOFTWARE (7th ed.). McGraw-Hill Interamericana de España S.L.

Sommerville, I. (2011). Ingeniería de software (9th ed.). Pearson Educación

Gómez Fuentes, M. del C., Cervantes Ojeda, J., & González Pérez, P. P. (Eds.). (2019). Fundamentos de ingeniería de software (1.ª ed.). Universidad Autónoma Metropolitana, Unidad Cuajimalpa.